

Decisiones de diseño y justificación

Objetivo

Este documento registra y justifica las decisiones de diseño tomadas en el sistema de inscripciones a actividades, con foco en claridad de responsabilidades, mantenibilidad y testabilidad (TDD).

Contexto y alcance

- Dominio: gestión de actividades con horarios y cupos, e inscripción de visitantes con validaciones de negocio.
- Alcance actual: almacenamiento en memoria, interfaz gráfica simple (Tkinter), y pruebas con pytest.

Visión general de la arquitectura

Capa UI (Tkinter) → Orquestación (GestorActividades) → Reglas de negocio (ServicioInscripcion) → Modelo de dominio (Actividad, Visitante) → DTO de resultado (ResultadoInscripcion)

- UI (main.py): recopila datos, invoca al Gestor y muestra mensajes. Evita lógica de negocio.
- GestorActividades: catálogo de actividades y punto de entrada de inscripciones a nivel de aplicación.
- ServicioInscripcion: centraliza y aplica reglas de negocio para validar y registrar inscripciones.
- Actividad: gestiona su estado interno (horarios, cupos, inscripciones) y reglas simples de consistencia.
- Visitante: datos del participante.
- ResultadoInscripcion: DTO explícito para comunicar éxito/fracaso y mensajes al usuario.

Principales decisiones y su justificación

1) Separación de responsabilidades (SRP)

- Por qué: reduce acoplamiento, facilita pruebas unitarias y evolución del código.
- Cómo: la UI no valida reglas de negocio; el ServicioInscripcion las centraliza; Actividad solo gestiona estado interno y consultas simples.

2) ServicioInscripcion como orquestador y validador

- Motivo: agrupar reglas de negocio en un único lugar permite reutilización y coherencia (evita duplicar validaciones en UI/entidades).
- Efecto: pruebas enfocadas (test_inscripcion.py) cubren escenarios de negocio sin depender de la UI.

3) Entidad Actividad con estado y reglas locales

- Responsabilidades: validar horarios existentes, calcular cupos disponibles, registrar inscripciones.
- Por qué: mantiene el modelo coherente sin mezclar reglas transversales (p. ej., términos y datos obligatorios) que pertenecen al Servicio.

4) DTO explícito ResultadoInscripcion

- Alternativa: excepciones para control de flujo.
- Decisión: usar un DTO con `exitoso` y `mensaje` :
 - Simplifica el manejo en la UI (mensajes al usuario).
 - Evita usar excepciones para casos esperables de validación.

5) Estructuras de datos simples y legibles

- Horarios como `Dict[str, int]` y claves string: fácil de mostrar en UI y suficiente para franjas horarias discretas ("10", "11", ...).
Alternativa (datetime/timedelta) se descartó para no sobredimensionar el problema actual.
- Inscripciones por horario como `List[Visitante]` : permite calcular disponibles con `cupos - len(lista)` de forma directa.

6) Validaciones de dominio explícitas

- Reglas implementadas en `ServicioInscripcion` :
 - Debe haber al menos un visitante y aceptar términos y condiciones.
 - Horario válido y cupos suficientes.
 - Datos del visitante completos y válidos: nombre (alfabético), DNI entero positivo, edad positiva, y talla si la actividad lo requiere.
- Justificación: estas reglas son transversales a la UI y a la persistencia, por lo que residen en el servicio de dominio/aplicación.

7) GestorActividades como catálogo central

- Mantiene un registro de actividades disponibles y delega la inscripción al servicio.
- Permite agregar nuevas actividades sin modificar la UI ni las reglas del servicio (OCP a nivel de catálogo).

8) Diseño guiado por pruebas (TDD)

- Los tests cubren casos felices y fallos esperados (actividad inexistente, horario inválido, sin cupos, datos faltantes, términos no aceptados, etc.).
- Beneficios: especificación ejecutable, refactorizaciones seguras y documentación viva del comportamiento.

Calidad y pruebas

- Pruebas unitarias con pytest y fixtures para aislar escenarios.
- Mensajes claros y trazables al origen de la regla fallida.
- El diseño facilita probar el servicio y la entidad sin levantar la UI.

Conclusión

El diseño prioriza separación clara de responsabilidades, simplicidad en las estructuras de datos y centralización de reglas en un servicio dedicado. Esto habilita pruebas efectivas y una UI simple.

Guía de estilo de código (Python)

Objetivos

- Mejorar legibilidad y mantenibilidad.
- Facilitar el trabajo en equipo (mismas reglas y expectativas).
- Evitar errores comunes mediante convenciones claras.

Estructura y responsabilidades

- Separación de capas:
 - Dominio/Negocio: `models.py` (modelos, validaciones, reglas de negocio).
 - Interfaz de usuario: `main.py` (Tkinter/ ttk, eventos, bindings, presentación).
 - Pruebas: `test_inscripcion.py` (pytest, AAA: Arrange-Act-Assert).
- La UI no debe contener reglas de negocio; delega en servicios/gestores (p. ej., `ServicioInscripcion` , `GestorActividades`).
- Evitar acceso directo a estructuras internas desde UI; preferir métodos públicos del gestor/servicio.

Nombrado (naming)

- Módulos/archivos: `snake_case` en minúsculas (ej.: `clases.py` , `test_inscripcion.py`).
- Clases: `PascalCase` (ej.: `GestorActividades` , `ServicioInscripcion` , `MenuPrincipal`).
- Funciones y métodos: `snake_case` (ej.: `registrar_inscripcion` , `obtener_actividad`).
- Variables y atributos: `snake_case` en español (ej.: `requiere_talla` , `acepta_terminos`).
- Constantes: `MAYÚSCULAS_CON_GUIONES_BAJOS` (ej.: `COLORES`).
- Fixtures y tests de pytest: `snake_case` (ej.: `actividad_con_cupos` , `visitante_sin_talla`).

Tipado y modelos de datos

- Usar type hints en funciones, métodos y atributos públicos.
- Usar `@dataclass` para DTOs/resultados simples (ej.: `ResultadoInscripcion`).
- Consistencia de tipos: alinear anotaciones con validaciones/uso real. Ej.: si `dni` se valida como entero positivo, el tipo a usar debe ser `int` en constructores, anotaciones y pruebas.

Docstrings y comentarios

- Docstrings en español, triple comilla, con breve descripción y secciones `Args:` / `Returns:` cuando aplique.
- Comentarios en línea sólo para aclarar decisiones no obvias. Evitar comentar lo evidente.
- En pruebas, documentar con bloques AAA para intención y claridad.

Ejemplo de docstring breve:

```
def es_horario_valido(self, horario: str) -> bool:
    """Indica si el horario pertenece a la actividad."""
    return horario in self.horarios
```

Imports

- Orden: estándar > terceros > locales. Separar por líneas en blanco.
- Preferir imports explícitos y consistentes en UI Tkinter:
 - `import tkinter as tk` para base.
 - `from tkinter import ttk, messagebox` para widgets/utilidades comunes.
- Evitar wildcard imports (`from x import *`).
- Usar `isort` para ordenar automáticamente.

Estilo y formato

- Seguir PEP 8.

Validación, errores y flujos de negocio

- Validaciones centralizadas en servicios (p. ej., `ServicioInscripcion`).
- Para flujos esperados de validación, devolver objetos resultado (`ResultadoInscripcion`) en lugar de lanzar excepciones.
- Mensajes de error/éxito en español, con mayúscula inicial y punto final.
- Reglas actuales (resumen):
 - Debe aceptarse términos y condiciones.
 - Debe haber al menos un visitante.
 - Horario debe ser válido para la actividad y con cupos suficientes.
 - Para cada visitante: nombre válido sin números ni caracteres especiales; DNI entero positivo; edad > 0; si la actividad lo requiere, talla ∈ {XS, S, M, L, XL}.
- Nota: actualmente los nombres se validan con `str.isalpha()` , lo que no permite espacios ni tildes combinadas; si se desea permitir nombres con espacios/tildes, actualizar esta regla y las pruebas asociadas.

Interfaz de usuario (Tkinter/ttk)

- Paleta de colores en constantes (ej.: `COLORES`) y reutilización consistente.
- Usar `ttk` para widgets modernos (combobox/spinbox) y `tk` cuando corresponda.
- Prefijar variables de estado con `*_var` (`StringVar` , `BooleanVar` , etc.).
- Handlers de eventos con nombres verbales claros (ej.: `actualizar_horarios` , `inscribir`).
- No mezclar lógica de negocio en handlers; delegar en `GestorActividades` / `ServicioInscripcion` .
- Mantener el layout legible: usar `grid` con `sticky` , `padx/pady` y separar secciones con `Frame` /separadores visuales.

Pruebas (pytest)

- Nombrar archivos de test como `test_*.py` y funciones `test_*`.
- Usar fixtures descriptivos y reutilizables (ej.: `actividad_con_cupos`).
- Estructurar cada test con AAA y comentarios:
 - `# ===== ARRANGE =====`
 - `# ===== ACT =====`
 - `# ===== ASSERT =====`
- Verificar efectos colaterales además del resultado (p. ej., cupos disponibles tras la inscripción).
- Asserts sobre mensajes: normalizar a `lower()` si se testea por inclusión de texto.
- Cubrir casos borde: horarios inválidos, sin cupos, sin aceptar términos, campos faltantes, etc.

Estándares de cadenas y localización

- Idioma por defecto: español (UI y mensajes de negocio).
- Mantener coherencia ortográfica (acentuación) y estilo (imperativos breves, afirmativos/negativos claros).
- Evitar hardcodear textos repetidos en muchos lugares; considerar constantes si se reutilizan.

Rendimiento y complejidad

- Mantener métodos con una sola responsabilidad.
- Evitar anidar demasiados bloques `if` / `for` en UI; extraer funciones auxiliares.
- En validaciones, devolver temprano ante fallos (early return) para simplicidad.

Registro y diagnóstico

- Para la UI, usar `messagebox` para feedback al usuario.

Compatibilidad y versiones

- Python 3.10+ recomendado (para mejoras de tipado). Ajustar si el entorno es distinto.
- Tkinter/ttk estándar; fuentes deben ser compatibles (en Windows, usar familias disponibles como "Segoe UI").